

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – Pencarian Nilai Max/Min



RASYA DIKA PRATAMA

109082500006

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrosesan data komputasional seringkali melibatkan operasi pencarian nilai tertinggi atau terendah dari sebuah kumpulan data. Dalam bahasa Go, wadah penyimpanannya bisa menggunakan struktur data array statis dengan kapasitas memori yang tetap, atau menggunakan tipe data slice yang ukurannya bisa disesuaikan secara dinamis saat program berjalan. Konsep pencarian ini tidak hanya terbatas pada tipe data dasar, tetapi juga bisa diterapkan pada tipe data bentukan (struct) yang mengelompokkan berbagai entitas berbeda ke dalam satu kesatuan objek.

Algoritma yang lazim digunakan untuk menemukan nilai ekstrem pada data yang tersusun secara acak adalah Sequential Search atau pencarian linear. Metode ini beroperasi dengan cara menjadikan elemen pada indeks awal sebagai nilai patokan sementara. Selanjutnya, program akan menelusuri elemen-elemen berikutnya satu demi satu untuk dibandingkan dengan patokan tersebut. Jika ditemukan nilai yang lebih ekstrem, maka patokan tersebut akan langsung diganti. Operasi ini berjalan dengan kompleksitas waktu $O(N)$. Untuk meningkatkan keterbacaan kode, logika ini sering dibungkus ke dalam subprogram terpisah. Jika fungsi tersebut perlu mengubah nilai asli dari suatu variabel secara langsung di dalam memori, maka digunakanlah mekanisme pass-by-reference dengan bantuan pointer.

B. Guided

1. Program Unguided 1

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
```

```

        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }

    return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }

    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int =
nilaiTerbesar(arrAngka[:])

    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])

```

```

        fmt.Printf("Nilai terkecil : %d ditemukan pada index
ke-%d", nilaiMin, idxNilaiMin)

        fmt.Println()

        fmt.Printf("Nilai terbesar : %d ditemukan pada index
ke-%d", nilaiMaks, idxNilaiMaks)

    }
}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2 main ✕
$ cd .\Week_9\Guided\Guided-1\

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Guided\Guided-1
$ go run .\contoh-1.go
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2

```

Penjelasan:

program ini dipakai buat nyari angka terbesar dan terkecil beserta posisinya dari sebuah *array* bilangan bulat. Logikanya simpel: ambil elemen pertama (indeks 0) sebagai patokan sementara, lalu lakukan *looping* untuk mengecek sisa datanya satu per satu. Kalau pas ngecek ketemu angka yang nilainya lebih ekstrem dari patokan, angka patokan tersebut bakal langsung ditimpa dengan nilai yang baru.

2. Program guided 2

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    IPK   float64
}

```

```

}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
    }
}

```

```

        fmt.Print("Masukkan NIM :")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=====")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
    fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim, idxMaks)
    fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim, idxMin)
}

```

Output :

```

Masukkan IPK : 3.3
=====
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : Dani
Masukkan NIM :10930032223
Masukkan IPK : 4.0
=====
DATA MAHASISWA INDEKS KE- 0
Nama : Dika
NIM : 1090302902
IPK : 3
=====
DATA MAHASISWA INDEKS KE- 1
Nama : jonathan
NIM : 10320930322
IPK : 3.3
=====
DATA MAHASISWA INDEKS KE- 2
Nama : Dani
NIM : 10930032223
IPK : 4
=====
=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 4.00, atas nama Dani dengan NIM 10930032223, ditemukan pada indeks ke-2IPK Terkecil : 3.00, atas nama Dika dengan NIM 1090302902, ditemukan pada indeks ke-0

```

Penjelasan:

Program ini versi *upgrade*-nya yang pakai *struct*. Tujuannya buat mencari data mahasiswa (lengkap dengan nama, NIM, dan IPK) yang punya nilai IPK paling tinggi dan paling rendah. Cara kerjanya persis kayak Guided 1, bedanya yang dibandingkan pas *looping* itu spesifik cuma di atribut *.IPK*-nya aja. Kalau ketemu IPK yang lebih ekstrem, satu paket data *struct* mahasiswa itu yang bakal ditimpa dan disimpan buat ditampilkan di akhir.

C. Unguided

1. Soal Unguided 1

- 1) Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
package main
```

```
import "fmt"

func main() {

    var n int

    var beratKelinci [1000]float64

    fmt.Scan(&n)

    if n <= 0 {

        return

    }

    for i := 0; i < n; i++ {

        fmt.Scan(&beratKelinci[i])

    }

    var min float64 = beratKelinci[0]
    var max float64 = beratKelinci[0]

    for i := 1; i < n; i++ {

        if beratKelinci[i] < min {

            min = beratKelinci[i]

        }

        if beratKelinci[i] > max {

            max = beratKelinci[i]

        }

    }

}
```



```
    }  
  
    }  
  
    fmt.Printf("%.2f %.2f\n", min, max)  
  
}
```

Output :

```
dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-1  
$ go run .\berat_kelinci.go  
3  
23 30 40  
23.00 40.00
```

Penjelasan :

Program bekerja dengan menyimpan seluruh input berat kelinci ke dalam satu array statis. Untuk menemukan nilai ekstrem, elemen di indeks ke-0 langsung diangkat sebagai "kandidat" nilai terkecil maupun terbesar. Setelah itu, program melakukan iterasi mulai dari indeks ke-1 hingga selesai, membandingkan setiap elemen dengan kandidat saat ini. Jika ada nilai yang lebih kecil atau lebih besar dari kandidat, variabel min atau max akan langsung diperbarui.

2. Soal Unguided 2

2) Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y . Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukkan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y , urutan ikan yang dimasukkan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import "fmt"

func main() {
    var x, y int
    var ikan [1000]float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&ikan[i])
    }
}
```

```
var totalWadah [1000]float64

var idxWadah int = 0

var beratSementara float64 = 0

var hitungIkan int = 0


for i := 0; i < x; i++ {

    beratSementara += ikan[i]

    hitungIkan++


    if hitungIkan == y || i == x-1 {

        totalWadah[idxWadah] = beratSementara

        idxWadah++

        beratSementara = 0

        hitungIkan = 0

    }

}


var totalSemua float64 = 0


for i := 0; i < idxWadah; i++ {

    fmt.Printf("%.2f", totalWadah[i])

    if i < idxWadah-1 {

        fmt.Print(" ")

    }

}
```

```

    }

    totalSemua += totalWadah[i]

}

fmt.Println()

var rataRata float64 = totalSemua / float64(idkWadah)

fmt.Printf("%.2f\n", rataRata)

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-2
$ go run .\tarif_ikan.go
4 3
2.5 3.5 4.0 1.0
10.00 1.00
5.50

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-2
$ go run .\tarif_ikan.go
5 2
1 2 3 4 5
3.00 7.00 5.00
5.00

```

Penjelasan :

Alih-alih menggunakan fungsi tambahan seperti append yang membuat ukuran array berubah dinamis, kode ini murni menggunakan array berukuran tetap dan sebuah counter (penghitung). Saat looping berjalan, berat ikan terus diakumulasikan ke variabel sementara. Begitu jumlah ikan mencapai batas kapasitas maksimal (y) atau tidak ada ikan tersisa, total berat tersebut dikunci ke dalam array totalWadah pada indeks tertentu, lalu counter diatur ulang ke nol untuk menghitung wadah berikutnya..

3. Soal Unguided 3

- 3) Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:

```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
  /* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
  ...
}

function rerata (arrBerat arrBalita) real {
  /* menghitung dan mengembalikan rerata berat balita dalam array */
  ...
}
```

Perhatikan sesi interaksi pada contoh berikut ini (teks bergaris bawah adalah input/read)

```
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```

Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {

    *bMin = arrBerat[0]

    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {

        if arrBerat[i] < *bMin {

            *bMin = arrBerat[i]

        }

        if arrBerat[i] > *bMax {

            *bMax = arrBerat[i]

        }

    }

}

func rerata(arrBerat arrBalita, n int) float64 {

    var total float64 = 0

    for i := 0; i < n; i++ {

        total += arrBerat[i]

    }

    return total / float64(n)

}
```

```

}

func main() {

    var n int

    var dataBalita arrBalita

    fmt.Print("Masukan banyak data berat balita: ")

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        fmt.Printf("Masukan berat balita ke-%d: ", i+1)

        fmt.Scan(&dataBalita[i])

    }

    if n > 0 {

        var min, max float64

        hitungMinMax(dataBalita, n, &min, &max)

        rata := rerata(dataBalita, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n", min)

        fmt.Printf("Berat balita maksimum: %.2f kg\n", max)

        fmt.Printf("Rerata berat balita: %.2f\n", rata)

    }

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-3
$ go run .\posyandu_balita.go
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 3.4
Masukan berat balita ke-2: 5
Masukan berat balita ke-3: 2.3
Masukan berat balita ke-4: 6
Berat balita minimum: 2.30 kg
Berat balita maksimum: 6.00 kg
Rerata berat balita: 4.17

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-3
$ go run .\posyandu_balita.go
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 3.2
Masukan berat balita ke-3: 4.5
Masukan berat balita ke-4: 9.4
Berat balita minimum: 3.20 kg
Berat balita maksimum: 9.40 kg
Rerata berat balita: 5.60

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_9\Unguided\Unguided-3
$

```

Penjelasan :

Program ini mendemonstrasikan pembagian tugas ke dalam subprogram.

Fungsi pencarian ekstrem memanfaatkan mekanisme pass-by-reference menggunakan pointer (*). Artinya, alih-alih mengembalikan (return) sebuah nilai tunggal ke fungsi utama, fungsi hitungMinMax akan secara langsung memodifikasi isi memori dari variabel min dan max aslinya. Sementara itu, fungsi penghitungan rata-rata tetap berjalan seperti biasa karena ia hanya bertugas menjumlahkan isi array dan mengembalikan satu hasil perhitungan matematika.

D. Kesimpulan

Dari praktikum ini, kesimpulannya adalah Linear Search merupakan metode yang sederhana tapi sangat bisa diandalkan untuk mencari nilai ekstrem pada kumpulan data acak. Trik terpentingnya selalu jadikan data pada elemen pertama sebagai patokan awal inisialisasi agar terhindar dari bug logika.

Selain itu, pemilihan struktur data sangat berpengaruh. Gunakan array untuk kapasitas data yang pasti, slice untuk perhitungan dinamis, dan struct untuk

mengelola objek dunia nyata yang lebih kompleks. Terakhir, kemampuan kita dalam memecah kode menjadi subprogram (modul) dan pemahaman tentang pointer memori adalah fondasi penting untuk membuat program yang efisien dan mudah dirawat.